

Temporary Table

Temporary Table (Bảng tạm thời)

- ❖ Là bảng chỉ tồn tại trong **phiên** làm việc hiện tại của kết nối đến MySQL.
- ❖ Khi kết nối đóng hoặc session kết thúc, bảng tạm này **cũng biến mất**.
- ❖ Có thể chứa dữ liệu giống như bảng thường.

```
CREATE TEMPORARY TABLE temp_table_name (  
    id INT,  
    name VARCHAR(100)  
);
```

```
DROP TEMPORARY TABLE temp_table_name
```

Temporary Table - Lưu ý

- ❖ You cannot refer to a **TEMPORARY** table more than once in the same query.

```
SELECT * FROM temp_table  
JOIN temp_table AS t2;
```

```
ERROR 1137: Can't reopen table: 'temp_table'
```

Khi nào cần dùng **Temporary Table** ?

- ❖ Khi cần lưu kết quả **trung gian** của **truy vấn phức tạp**.
- ❖ Khi cần lọc hoặc xử lý dữ liệu tạm thời mà không muốn tạo bảng thật trong database.

```
CREATE TEMPORARY TABLE temp_orders AS
SELECT customer_id, SUM(total_price) AS total
FROM orders
WHERE order_date > '2024-01-01'
GROUP BY customer_id;

SELECT * FROM temp_orders WHERE total > 1000;
```

View

View (Khung nhìn)

- ❖ View là **một truy vấn ảo**, không lưu dữ liệu thật mà chỉ lưu **định nghĩa truy vấn**.
- ❖ Khi gọi SELECT từ view, MySQL sẽ **thực thi truy vấn gốc**.

```
CREATE VIEW active_users AS  
SELECT id, name, email  
FROM users WHERE active = 1;
```

```
CREATE VIEW public_employees AS  
SELECT id, name, department FROM employees;
```

```
SELECT * FROM active_users;
```

Khi nào cần dùng View ?

- ❖ Khi muốn **tái sử dụng** truy vấn phức tạp ở nhiều nơi.
- ❖ Khi muốn **giấu logic truy vấn** khỏi người dùng cuối.
- ❖ Để tạo giao diện đọc dữ liệu **đơn giản** cho các ứng dụng.
- ❖ Khi muốn tạo bảng logic từ **nhiều bảng vật lý**.

Temporary Table VS View

Đặc điểm	Temporary Table	View
Lưu dữ liệu thật	Có	Không
Thời gian tồn tại	Trong 1 phiên (session)	Tồn tại lâu dài trong schema
Dùng trong truy vấn	Có	Có
Mục đích	Xử lý truy vấn tạm thời	Trừu tượng hóa đối tượng. Ẩn logic hoặc bảo mật
Dễ dàng thay đổi	Có, sử dụng UPDATE, DELETE, INSERT, ...	Khó hơn

Common Table Expression (CTE)

CTE

- ❖ **Common Table Expression (CTE)** – biểu thức bảng chung. CTE gần giống như **Temporary Table** nhưng chỉ dùng trong phạm vi của **một truy vấn** duy nhất.

```
WITH temp_table AS (  
    SELECT customer_id, SUM(total) AS total_spent  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT c.name, t.total_spent  
FROM temp_table t  
JOIN customers c ON t.customer_id = c.id  
WHERE t.total_spent > 10000000;
```

Khi nào cần dùng CTE ?

- ❖ Là cách đặt **tên tạm** cho một truy vấn con.
- ❖ Dễ đọc, dễ tái sử dụng trong cùng một câu lệnh.
- ❖ **Không** tạo bảng thật cũng **không** lưu dữ liệu.
- ❖ Khi không cần tái sử dụng sau đó.
- ❖ Cần viết nhiều tầng truy vấn lồng nhau mà không bị rối.

```
WITH customer_orders AS (  
    SELECT  
        customer_id,  
        COUNT(*) AS total_orders,  
        SUM(amount) AS total_spent,  
        MAX(order_date) AS last_order_date  
    FROM orders  
    GROUP BY customer_id  
) ,  
qualified_customers AS (  
    SELECT *  
    FROM customer_orders  
    WHERE total_orders > 2  
        AND total_spent > 500  
        AND last_order_date > '2024-01-01'  
)
```

```
SELECT  
    c.id,  
    c.name,  
    q.total_orders,  
    q.total_spent,  
    q.last_order_date  
FROM qualified_customers q  
JOIN customers c ON c.id =  
q.customer_id;
```

Use case

https://sql.zcode.vn/contests/CSDL_SQL/questions/b89cf4c90f4d489b81c95e7f861d58e9

Use case - Cách thông thường

```
select P.productId, P.productName, P.categoryId, SUM(OD.unitPrice * OD.quantity) as Revenue
from Product P
join OrderDetail OD
on OD.productId = P.productId
group by P.productId, P.productName, P.categoryId
having Revenue = (
    select max(Total) from (
        select P.productId, P.categoryId, SUM(OD2.unitPrice * OD2.quantity) as Total
        from (select * from Product P2 where P2.categoryId = P.categoryId) P2
        join OrderDetail OD2
        on OD2.productId = P2.productId
        group by P2.productId, P2.categoryId
    ) T2
)
ORDER BY P.categoryId;
```

Use case - Sử dụng CTE

```
WITH ProductRevenue AS (  
    SELECT  
        P.productId,  
        P.productName,  
        P.categoryId,  
        SUM(OD.unitPrice * OD.quantity) AS Revenue  
    FROM Product P  
    JOIN OrderDetail OD ON P.productId = OD.productId  
    GROUP BY P.productId, P.productName, P.categoryId  
) , MaxRevenuePerCategory AS (  
    SELECT  
        categoryId,  
        MAX(Rvenue) AS MaxRevenue  
    FROM ProductRevenue  
    GROUP BY categoryId  
)
```

```
SELECT  
    PR.productId,  
    PR.productName,  
    PR.categoryId,  
    PR.Revenue  
FROM ProductRevenue PR  
JOIN MaxRevenuePerCategory MRC  
    ON PR.categoryId = MRC.categoryId  
    AND PR.Revenue = MRC.MaxRevenue  
ORDER BY PR.categoryId;
```

Use case - Sử dụng Temporary Table

-- Tạo bảng tạm tính doanh thu sản phẩm

```
CREATE TEMPORARY TABLE TempProductRevenue AS
SELECT
    P.productId,
    P.productName,
    P.categoryId,
    SUM(OD.unitPrice * OD.quantity) AS Revenue
FROM Product P
JOIN OrderDetail OD ON P.productId = OD.productId
GROUP BY P.productId, P.productName, P.categoryId;
```

CREATE TEMPORARY TABLE TempProductRevenue2 AS

```
SELECT
    P.productId,
    P.productName,
    P.categoryId,
    SUM(OD.unitPrice * OD.quantity) AS Revenue
FROM Product P
JOIN OrderDetail OD ON P.productId = OD.productId
GROUP BY P.productId, P.productName, P.categoryId;
```

-- Truy vấn sản phẩm có doanh thu cao nhất trong từng danh mục

```
SELECT
    T1.productId,
    T1.productName,
    T1.categoryId,
    T1.Revenue
FROM TempProductRevenue as T1
JOIN (
    SELECT
        categoryId,
        MAX(Revenue) AS MaxRevenue
    FROM TempProductRevenue2
    GROUP BY categoryId
) T2
ON T1.categoryId = T2.categoryId
AND T1.Revenue = T2.MaxRevenue
ORDER BY T1.categoryId;
```

Use case - Sử dụng VIEW

```
-- Tạo view doanh thu sản phẩm
CREATE VIEW ProductRevenueView AS
SELECT
    P.productId,
    P.productName,
    P.categoryId,
    SUM(OD.unitPrice * OD.quantity) AS Revenue
FROM Product P
JOIN OrderDetail OD ON P.productId =
OD.productId
GROUP BY P.productId, P.productName,
P.categoryId;
```

```
-- Truy vấn từ view
SELECT
    PR.productId,
    PR.productName,
    PR.categoryId,
    PR.Revenue
FROM ProductRevenueView PR
JOIN (
    SELECT
        categoryId, MAX(Revenue) AS MaxRevenue
    FROM ProductRevenueView
    GROUP BY categoryId
) MR
ON PR.categoryId = MR.categoryId
AND PR.Revenue = MR.MaxRevenue
ORDER BY PR.categoryId;
```